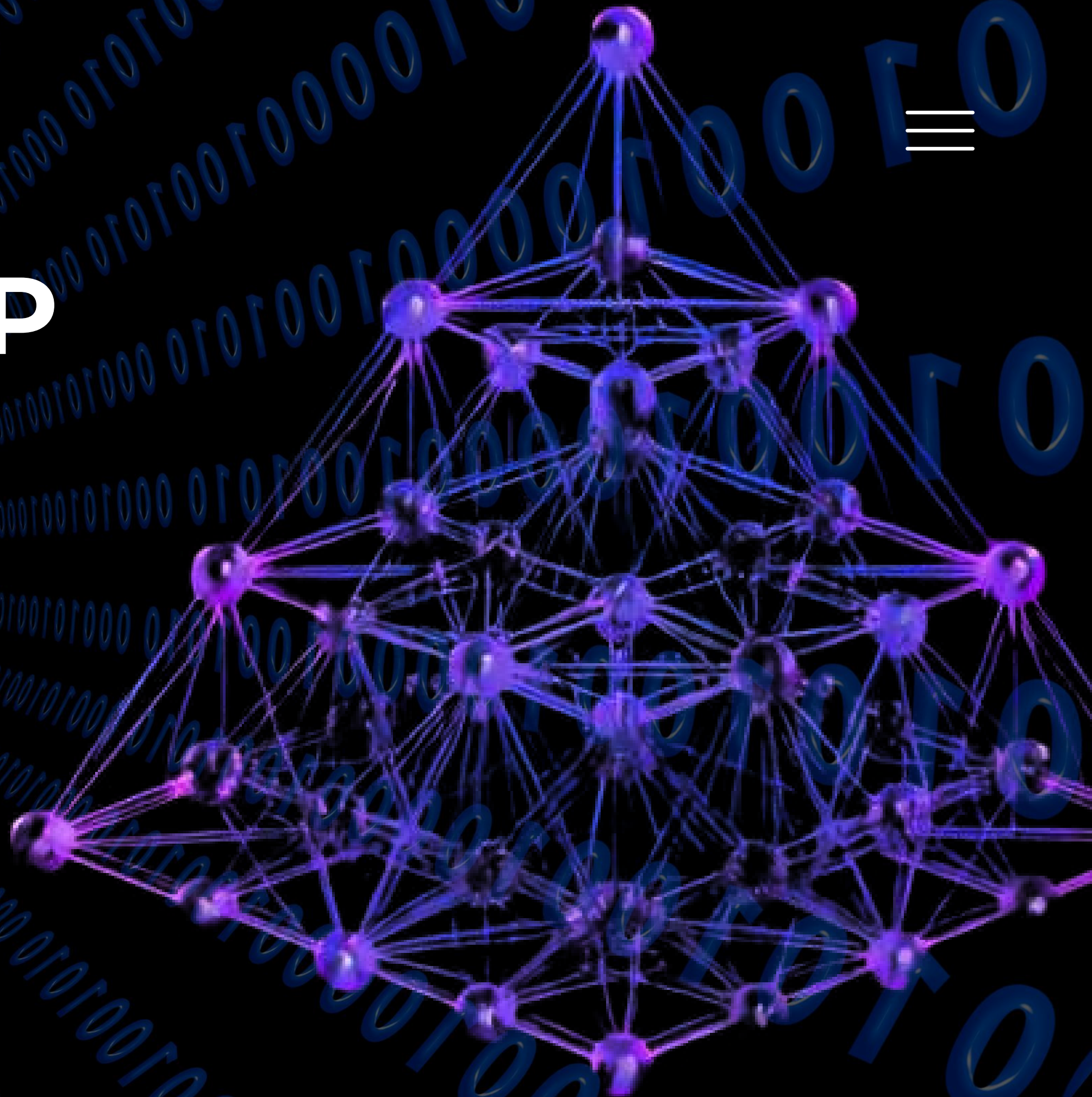


# HEAPS, MIN HEAP, MAX HEAP E HEAP SORT

ESTRUTURAS DE  
DADOS II

ALUNOS: EMERSON SOUZA, JACKSON VITÓRIO,  
MARJORY LETÍCIA, TAÍNÁ TARCILA,  
THAYNÁ FORTUNATO, TULIO LINS,  
WENDRIELE CLARA

PROFESSOR: TIAGO PESSOA LIMA





---

---

---

- ## Estruturas de Dados II



# Conceitos Fundamentais de Heap



## Min Heap

No Min Heap, o elemento da raiz é sempre o menor valor da árvore. Cada nó pai possui valor menor ou igual ao de seus filhos.

Propriedade:  $\text{pai}(i) \leq \text{filho}(i)$

Aplicações: filas de prioridade mínima, algoritmo de Dijkstra, extração eficiente do menor elemento em  $O(\log n)$ .

## Max Heap

No Max Heap, o elemento da raiz é sempre o maior valor da árvore. Cada nó pai possui valor maior ou igual ao de seus filhos.

Propriedade:  $\text{pai}(i) \geq \text{filho}(i)$

Aplicações: filas de prioridade máxima, Heap Sort, extração eficiente do maior elemento em  $O(\log n)$ .



# Representação de Heap em Vetor

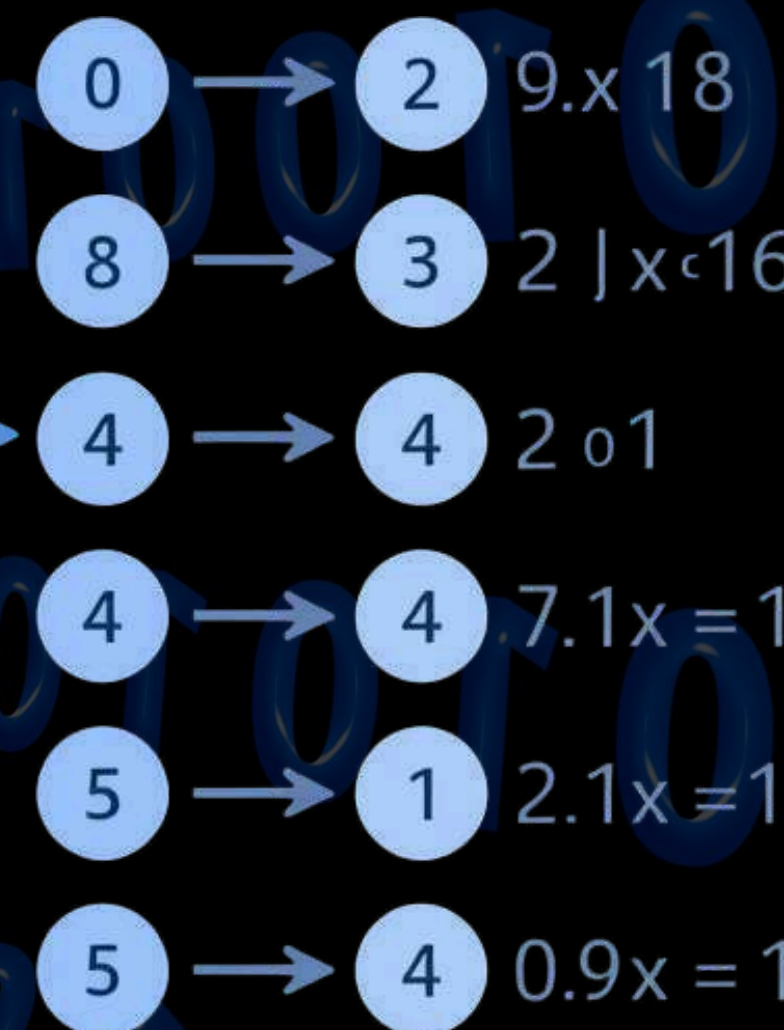
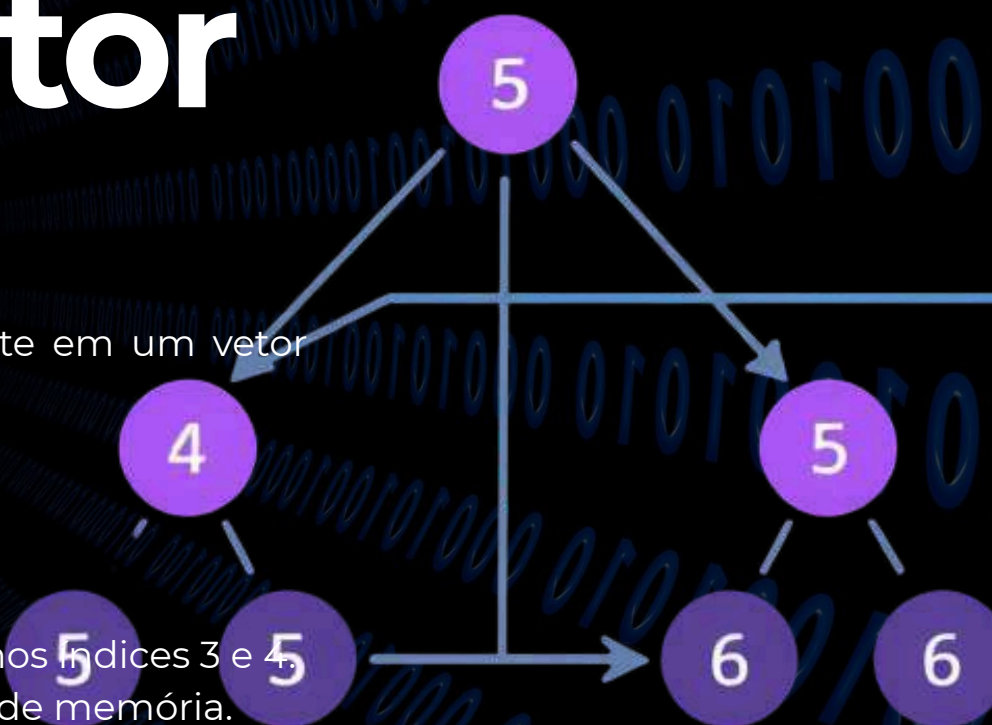
A árvore binária de um heap pode ser armazenada eficientemente em um vetor (array), sem uso de ponteiros.

Fórmulas de navegação por índice:

- $\text{Pai}(i) = (i - 1) / 2$
- $\text{Filho Esquerdo}(i) = 2i + 1$
- $\text{Filho Direito}(i) = 2i + 2$

Exemplo: raiz no índice 0; seus filhos nos índices 1 e 2; filhos do nó 1 nos índices 3 e 4.

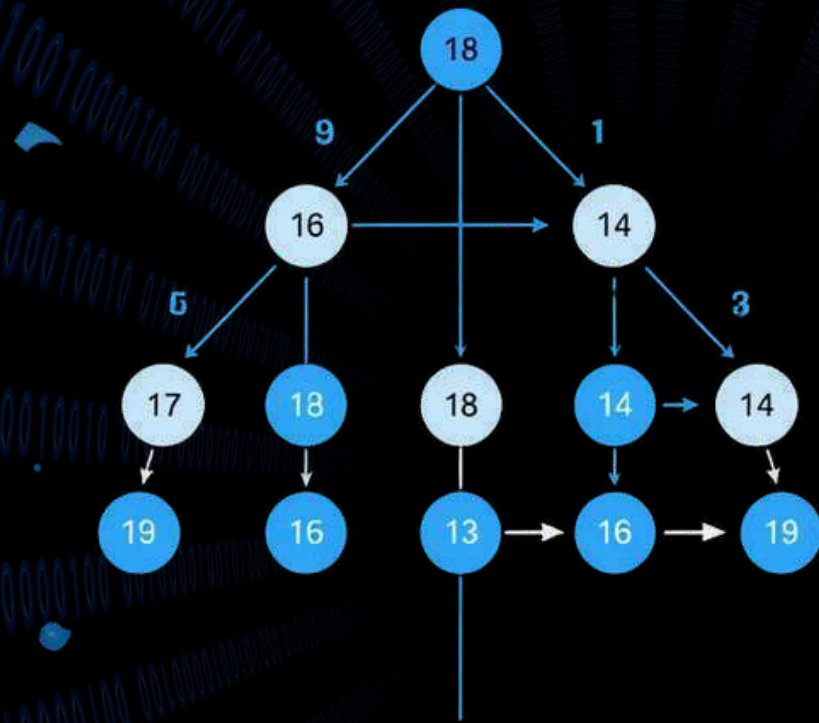
Vantagem: acesso direto  $O(1)$  a qualquer nó sem alocação dinâmica de memória.





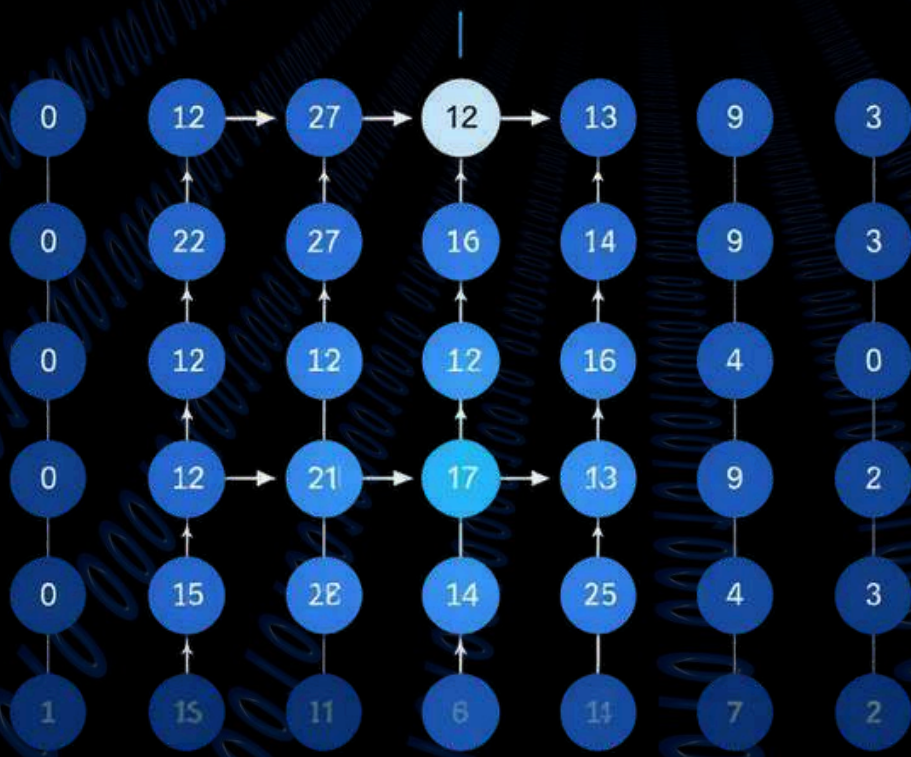
==

## Heap Insertion operation



## Operação de Inserção em Heap

Parent ind up fus to ksies ave min-heapprovers.



Passos da inserção em um Min Heap:

1. Adicionar o novo elemento ao final do vetor (última posição disponível).
2. Aplicar Heapify-Up: comparar o elemento inserido com seu pai. Se for menor (Min Heap), trocar as posições.
3. Repetir o processo subindo na árvore até que a propriedade do heap seja restaurada ou a raiz seja atingida.

Complexidade:  $O(\log n)$  — a altura da árvore limita o número de comparações.



#Max-Heap 2 →

Thev liring wonthave on-learn  
alsid o tean vou lífr down.

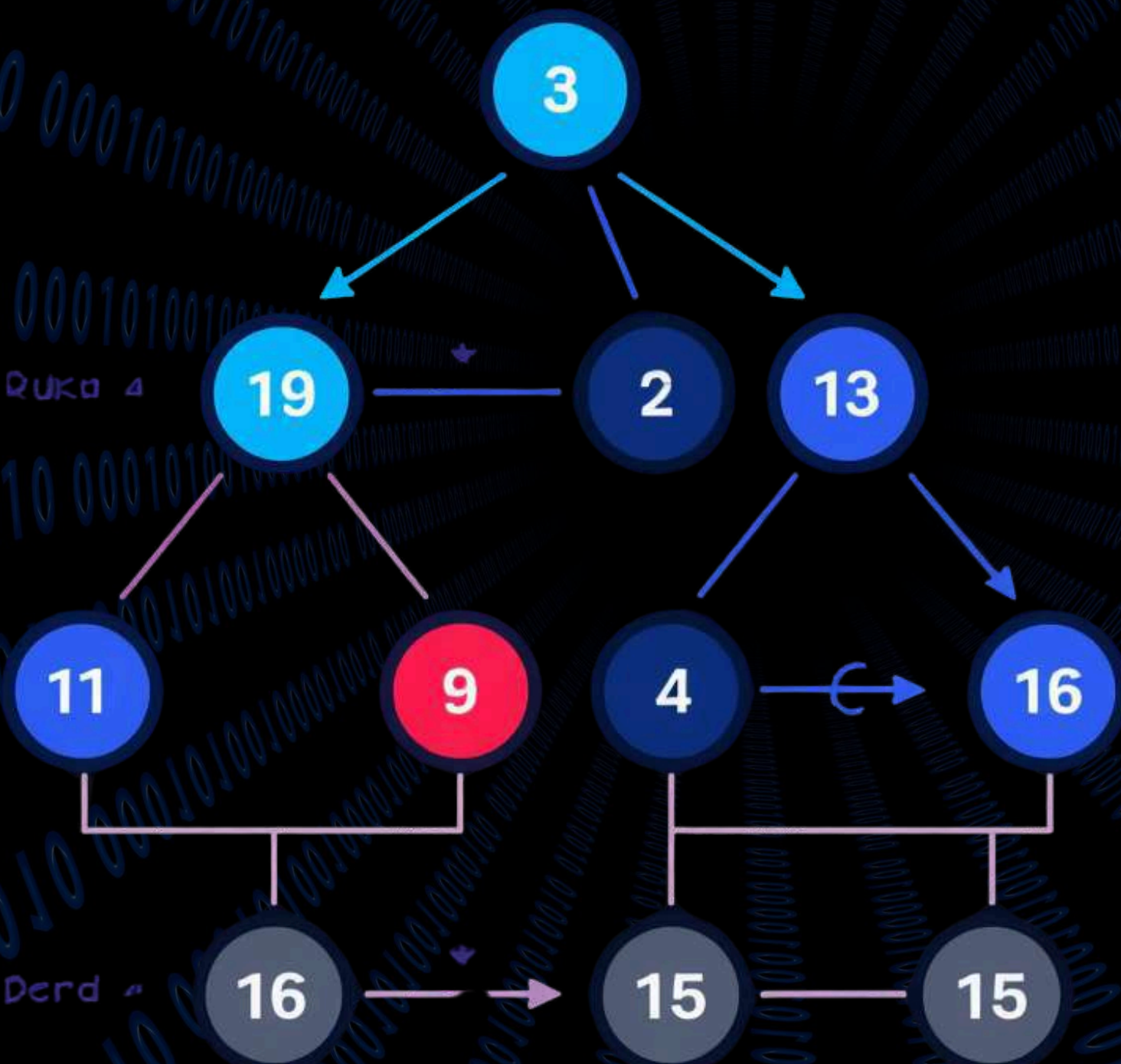
==

# Operação de Remoção em Heap

Passos da remoção no Max Heap:

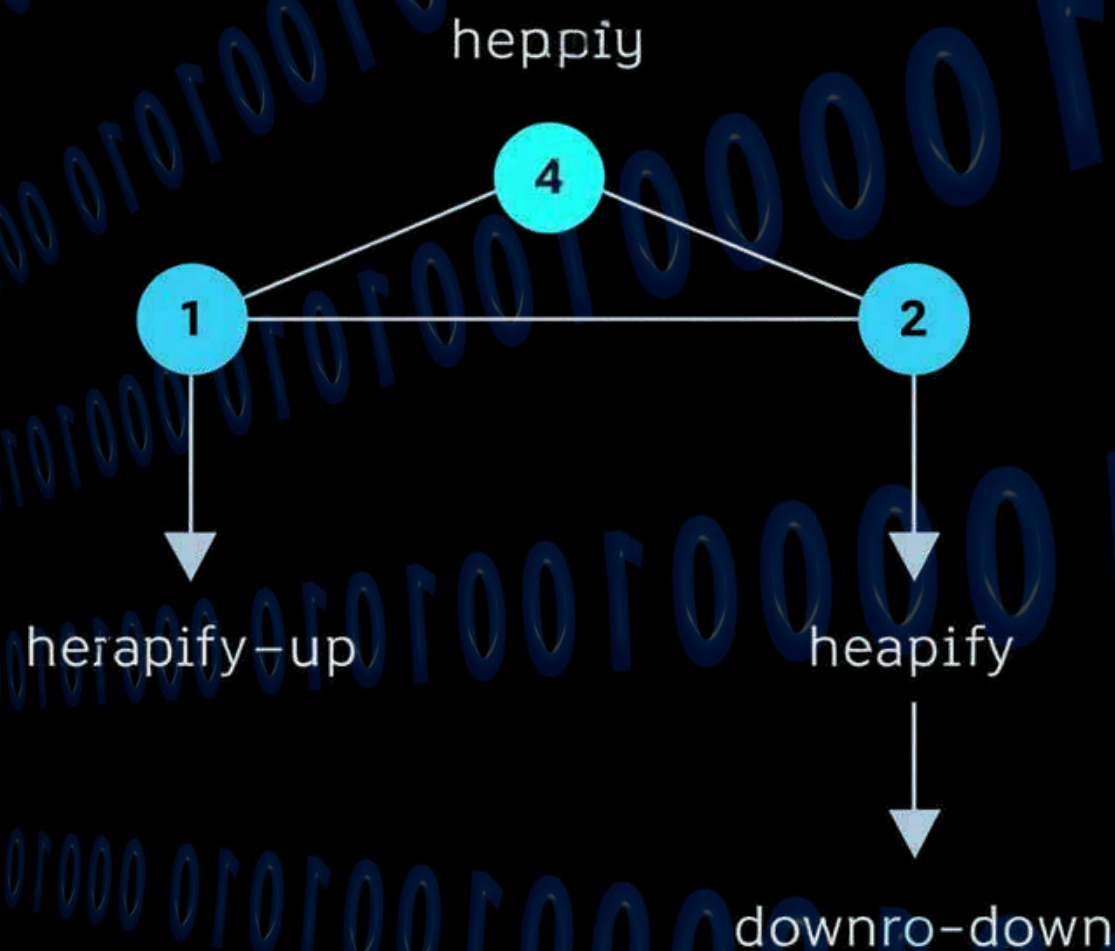
1. Substituir a raiz pelo último elemento do vetor
2. Remover o último elemento (reduzir tamanho do heap)
3. Aplicar heapify-down a partir da raiz
4. Comparar nó com filhos; trocar com o maior filho se necessário
5. Repetir até restaurar a propriedade do Max Heap

Complexidade:  $O(\log n)$





# Operação Heapify



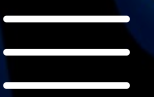
## Heapify-Up

Aplicado após inserção de um novo elemento. O nó recém-inserido sobe na árvore enquanto violar a propriedade do heap. Compara o nó com seu pai: se menor (Min Heap) ou maior (Max Heap), troca as posições. Repete até a raiz ou até a propriedade ser satisfeita. Complexidade:  $O(\log n)$ .

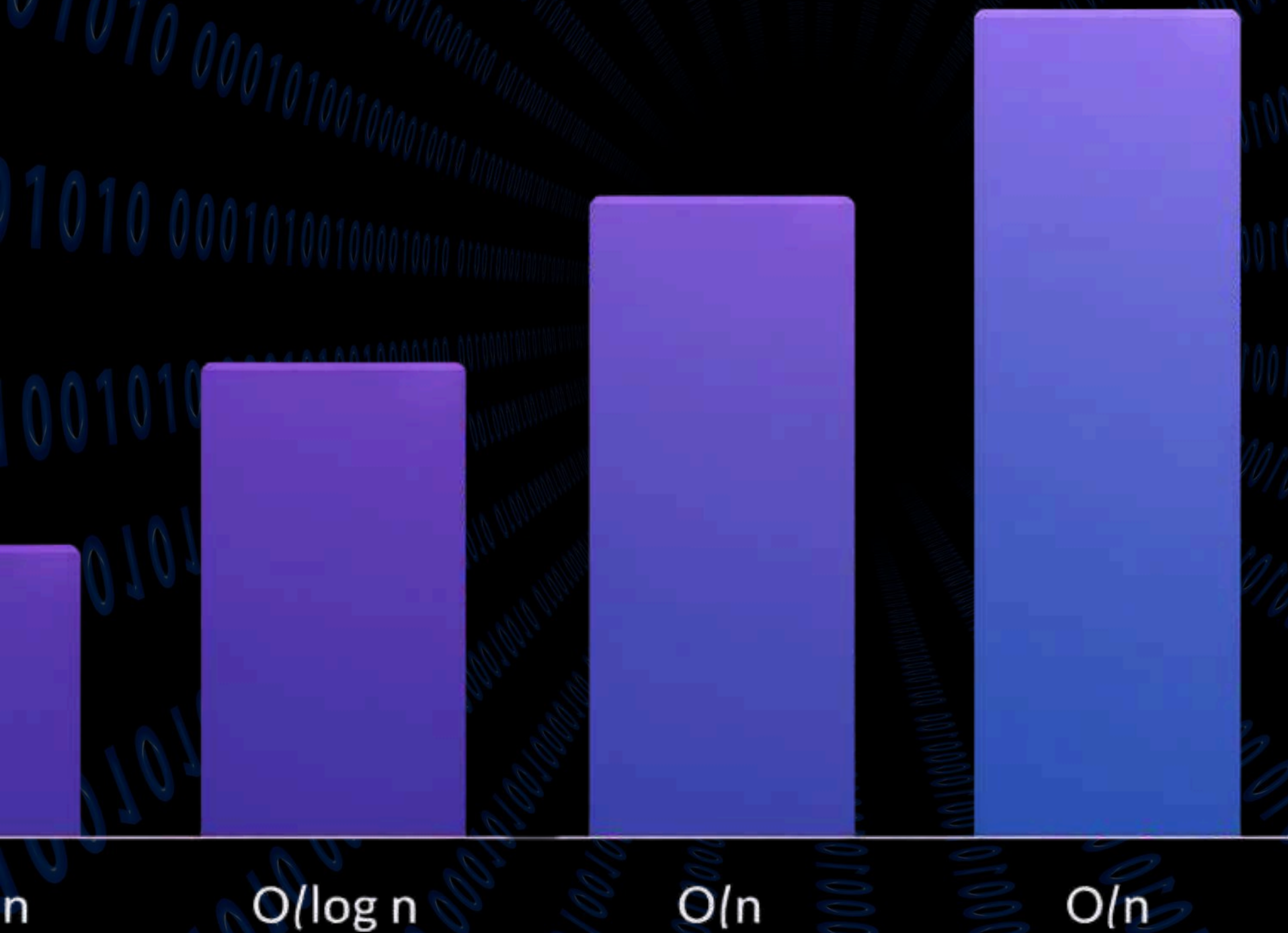
## Heapify-Down

Aplicado após remoção da raiz. O nó substituto desce na árvore enquanto violar a propriedade do heap. Compara o nó com seus filhos: troca com o menor filho (Min Heap) ou maior filho (Max Heap). Repete até as folhas ou até a propriedade ser restaurada. Complexidade:  $O(\log n)$ .





# Análise de Complexidade



- Inserção:  $O(\log n)$  — sobe pelo heap (heapify-up) até restaurar a propriedade.
  - Remoção (raiz):  $O(\log n)$  — desce pelo heap (heapify-down) após substituição.
  - Heapify:  $O(\log n)$  — ajuste de um nó em subárvore de altura  $\log n$ .
  - Build Heap:  $O(n)$  — construção eficiente aplicando heapify de baixo para cima.
- A altura de um heap com  $n$  elementos é  $\lceil \log_2 n \rceil$ , o que garante eficiência logarítmica nas operações principais.

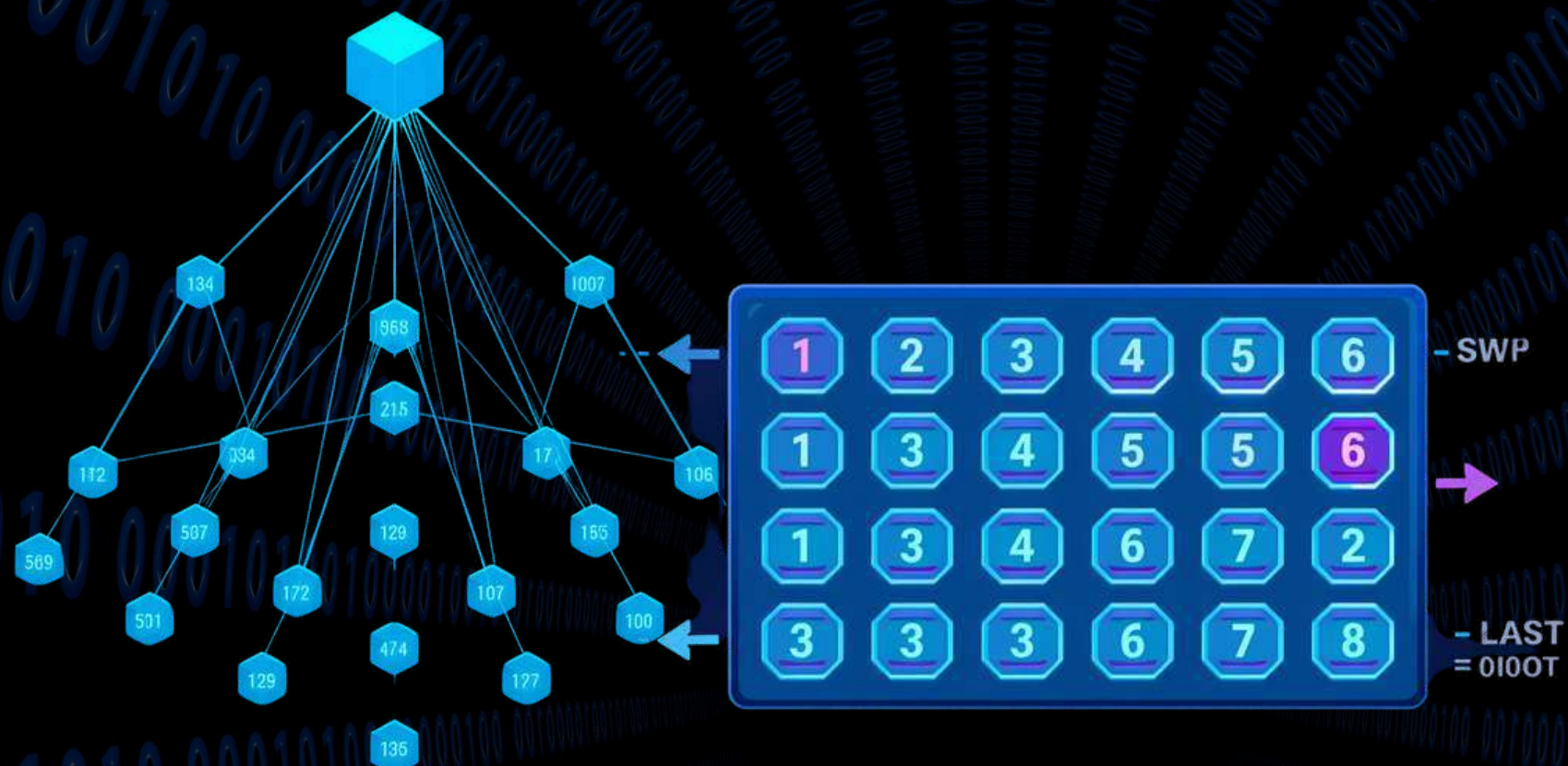




# Heap Sort Introdução

O Heap Sort é um algoritmo de ordenação eficiente baseado na estrutura de Max Heap. Seus passos gerais incluem: (1) construir um Max Heap a partir do array de entrada; (2) extrair repetidamente o elemento raiz (maior valor) e reposicioná-lo ao final do array; (3) reaplicar o heapify para restaurar a propriedade do heap. Vantagens: complexidade  $O(n \log n)$  em todos os casos, ordenação in-place ( $O(1)$  de espaço auxiliar) e desempenho consistente independente da entrada.





# Funcionamento do Heap Sort

## 1. Construir Max Heap

Dado o vetor de entrada, aplica-se o processo de build heap. Percorre-se o vetor de trás para frente a partir do último nó interno, aplicando heapify-down em cada posição. Ao final, o maior elemento estará na raiz (índice 0) do array.

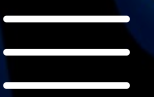
## 1. Trocar e Reduzir

O elemento da raiz (maior valor) é trocado com o último elemento do heap. O tamanho do heap é reduzido em 1, excluindo o elemento já ordenado. Repete-se n-1 vezes até que todos os elementos estejam em sua posição correta.

## 1. Reaplicar Heapify

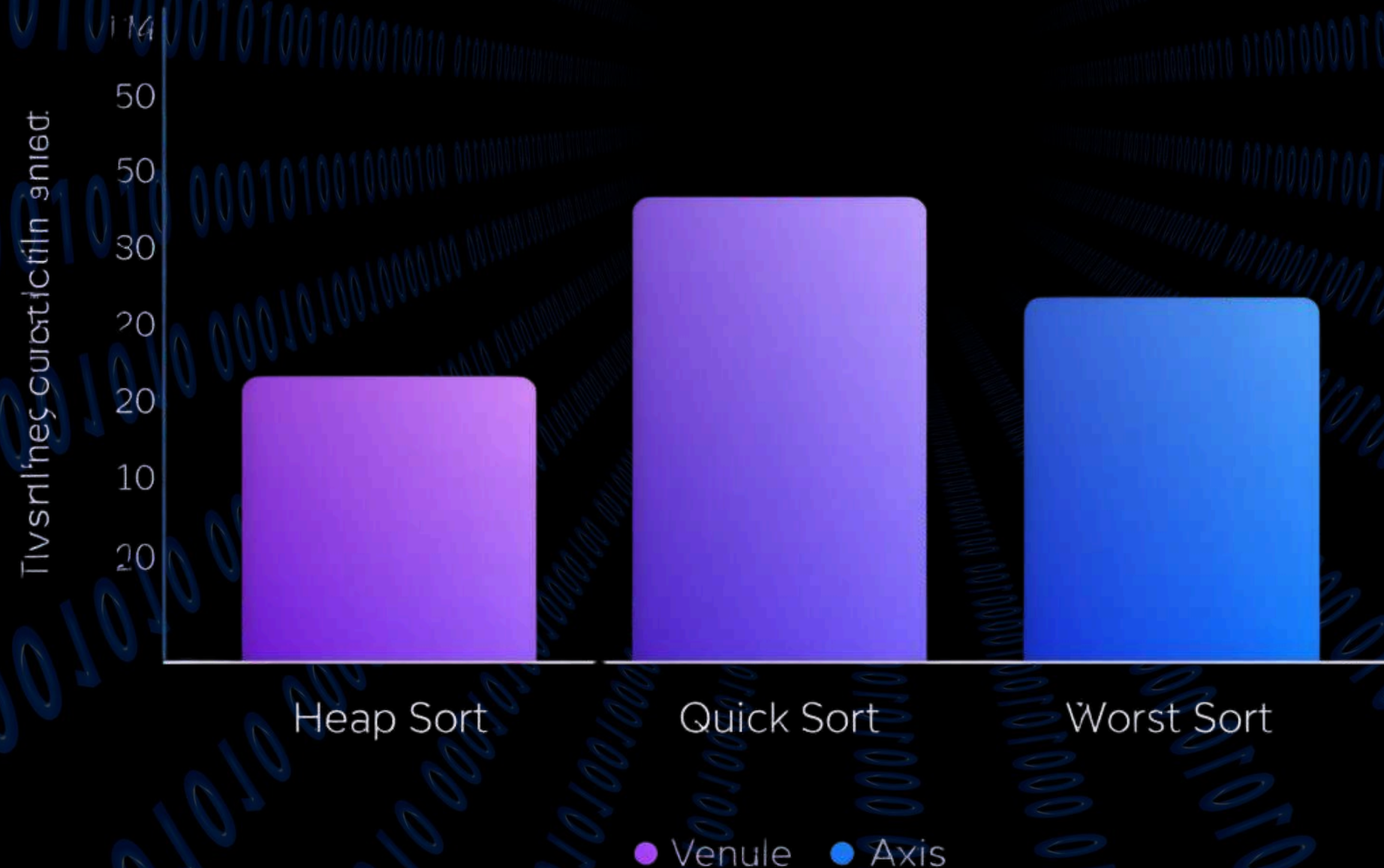
Após cada troca, aplica-se heapify-down na nova raiz para restaurar a propriedade de max heap no subvetor restante. O processo garante que o próximo maior elemento suba à raiz, mantendo a ordenação crescente no final do vetor.





Time complexity of sorting algorithms

# Análise de Complexidade do Heap Sort



Heap Sort apresenta desempenho  $O(n \log n)$  nos casos melhor, médio e pior — garantia que Quick Sort não oferece. Utiliza espaço  $O(1)$  in-place, sem memória auxiliar. Quick Sort tem  $O(n^2)$  no pior caso; Merge Sort exige  $O(n)$  de espaço extra. Heap Sort é ideal quando memória é limitada e consistência de tempo é essencial.



# Aplicações Práticas dos Heaps



Filas de Prioridade em SO e Redes  
Heaps são a base das filas de prioridade usadas em sistemas operacionais para escalonamento de processos e em redes para gerenciamento de pacotes, garantindo que tarefas mais urgentes sejam processadas primeiro com eficiência  $O(\log n)$ .



Algoritmos de Grafos (Dijkstra, Prim)  
Os algoritmos de Dijkstra (menor caminho) e Prim (árvore geradora mínima) utilizam heaps (min-heaps) para selecionar eficientemente o próximo vértice de menor custo, reduzindo a complexidade total para  $O((V + E) \log V)$ .



Gerenciamento de Tarefas e Eventos  
Simuladores de eventos discretos e gerenciadores de tarefas usam heaps para processar eventos em ordem cronológica ou por prioridade, permitindo inserção e remoção eficiente em aplicações de tempo real e jogos.



# Resumo e Considerações Finais

- Heaps são árvores binárias com propriedade de ordem: Min Heap (raiz = menor) e Max Heap (raiz = maior).
- Representação eficiente em vetor com fórmulas de navegação:  $\text{pai}(i) = (i-1)/2$ , filhos em  $2i+1$  e  $2i+2$ .
- Operações de inserção e remoção com complexidade  $O(\log n)$ ; construção de heap em  $O(n)$ .
- Heap Sort: algoritmo in-place  $O(n \log n)$  nos casos melhor, médio e pior.
- Aplicações práticas: filas de prioridade, algoritmos de grafos (Dijkstra, Prim) e escalonamento de tarefas.





# Referências Bibliográficas

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

KNUTH, D. E. The Art of Computer Programming, Vol. 3: Sorting and Searching. 2. ed. Addison-Wesley, 1998.

SEGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley Professional, 2011.

GOODRICH, M. T.; TAMASSIA, R. Estruturas de Dados e Algoritmos em Java. 5. ed. Bookman, 2013.

Recursos Online: GeeksforGeeks — Heap Data Structure: [geeksforgeeks.org/heap-data-structure/](https://www.geeksforgeeks.org/heap-data-structure/) | Visualgo — Visualização de Heaps: [visualgo.net/en/heap](https://visualgo.net/en/heap) | NIST Dictionary of Algorithms: [xlinux.nist.gov/dads/](https://xlinux.nist.gov/dads/) | MIT OpenCourseWare — Introduction to Algorithms: [ocw.mit.edu](https://ocw.mit.edu)



# OBRIGADO!

DÚVIDAS E DISCUSSÕES  
SÃO BEM-VINDAS!

## Estruturas de Dados II

Heaps, Min Heap, Max Heap e Heap Sort — conceitos fundamentais para algoritmos eficientes. Entre em contato para aprofundar o estudo sobre estruturas de dados.

